CHAI-FP: Characterizing Hierarchical AI Model Fingerprints from Network Traffic Metadata

Lawrence Oyaniyi*, Munmun Talukder†, Anik Mallik*, Moinul Hossain‡, Wei Yu*
*Department of Computer & Information Sciences, Towson University, MD, USA
†Department of Computer Science and Engineering, Wright State University, OH, USA
‡Department of Cyber Security Engineering, George Mason University, VA, USA

Abstract—Federated learning (FL) keeps training data on client devices, but the network traffic it generates can still reveal which model is being trained. We present CHAI-FP, a framework that characterizes how much AI model identity a passive observer can infer from packet-level metadata alone, without inspecting payloads or model parameters. CHAI-FP reconstructs the round structure of FL traffic and identifies models hierarchically at the family, architecture, and variant levels. Its key element is cross-device fingerprinting. Because FL clients train the same model on heterogeneous hardware and operating systems, the differences between their traffic reveal the architecture's compute profile. On a physical testbed with three heterogeneous clients (two Ubuntu, one Windows) and 32 experiments spanning convolutional, recurrent, transformer, and autoencoder models, cross-device fingerprinting identifies the family of unseen models with a balanced accuracy of 0.74, compared with 0.41 for single-client classification, and raises family accuracy in 5 s windows from 0.31 to 0.51. Within a family, the architecture is identified with a mean balanced accuracy of 0.79 from complete traces, and convolutional architectures with 0.71 from 5 s windows. The model payload size is a baseline leak that reveals the exact variant in every trace, and in 69% from a single model download within a 5 s window, but it is removed by padding, whereas family identification from timing alone remains at 0.62, showing that concealing model identity in FL requires timing defenses in addition to size padding.

Index Terms—Federated learning, traffic analysis, side channels, model fingerprinting, network security, privacy.

I. Introduction

Artificial intelligence (AI) workloads are increasingly executed across edge devices, cloud services, and distributed learning systems [1], [2]. Although encryption and access controls protect transmitted content and model artifacts, they do not conceal all characteristics of the communication generated during execution: packet sizes, directions, timing, volumes, and burst patterns remain observable to a network intermediary [3]. Such metadata may reveal properties of an AI workload even when the observer cannot inspect model parameters, application data, or payloads [4]. Federated learning (FL) is particularly exposed, because every training round produces a characteristic exchange in which each client downloads the global model, trains locally, and uploads an update.

Existing Research: Prior studies infer neural network information from indirect execution observations, including memory accesses, shared caches, GPU execution traces, and system resource measurements [5]–[8]. These observations require physical access to, or co-location with, the executing device. Network traffic is a more remote observation surface, on which communication protocols, training schedules, transport behavior, and network conditions may obscure architectural differences. Encrypted-traffic analysis reveals visited websites [3], [9]–[12], and PACKETPRINT [13] identifies mobile applications and user activities from encrypted wireless packet sequences, although its hierarchy captures traffic patterns rather than structural relationships among AI models. For AI workloads, network traffic has been used to discriminate CNN and RNN families in FL [4], to classify CNN and RNN architectures and variants from encrypted wireless traffic with results reported primarily at the family level [14], to identify AI applications and their execution phases [15], and to identify AI families from 5G scheduling metadata [16].

Gap: It is not established whether a passive observer can progressively distinguish families, architectures, and variants, how much of this information short windows reveal, or whether it survives size padding. Existing approaches also classify each client independently and ignore that all FL clients train the same model on heterogeneous hardware and operating systems.

Motivation: Different models may produce similar traffic under common FL protocols, short windows capture only part of a round, and observations of one execution are correlated, so generalization must be assessed on models absent from training. Knowing a model's architecture also helps an adversary, for example by training a matching surrogate for model extraction or transfer attacks.

This Work: We present CHAI-FP, a conditional hierarchical framework that infers the family, architecture, and variant of the model under training from passive packet metadata. CHAI-FP reconstructs the round structure of FL traffic, from which it derives communication features and a passive estimate of local training time, the compute gap between model download and upload, and it combines the traffic of all clients into a cross-device representation. We evaluate it on a physical FL testbed with three clients that differ in both hardware and operating system, and 32 experiments, using model-disjoint validation on 0.5 to 5 s windows and complete traces.

Contributions: (i) A conditional hierarchical framework that identifies model families, architectures, and variants from passive metadata, combining packet-morphology and compute-timing features; architectures are identified with a mean balanced accuracy of 0.79. (ii) Cross-device fingerprinting, which raises family identification of unseen models from 0.41 to 0.74 for complete traces and from 0.31 to 0.51 for 5 s windows; an ablation shows that the gain requires both the cross-device representation and a regularized linear classifier. (iii) A model-disjoint evaluation showing that payload size reveals the exact variant in every trace but is removed by padding, whereas family identification from timing alone remains at 0.62, so concealing model identity requires timing defenses.

II. Threat Model

We consider a passive adversary on the network path at the aggregation proxy of an FL deployment, such as a compromised switch, a network tap, or an honest-but-curious proxy. For each client connection, the adversary records the timestamp, direction, length, and TCP flags of every packet, but never inspects payloads and never injects, drops, or modifies traffic; these quantities are available whether or not the payload is encrypted. Its goal is to recover the family, architecture, and variant of the model under training from this metadata alone.

III. Experimental Setup

The experimental testbed consists of three AI execution clients, a coordinating server, and a passive network proxy positioned along the communication path (Sec. II). The testbed is heterogeneous in both hardware and operating system: the clients are an NVIDIA Jetson Orin Nano and a Dell desktop running Ubuntu, and a Dell laptop running Windows 11, whereas the server, hosted on an NVIDIA Jetson AGX Orin, and the proxy run Ubuntu. The clients therefore differ in compute capability as well as in their network stacks; for example, the Windows client omits the TCP timestamp option that the Ubuntu hosts use by default. In two experiments, the Jetson AGX Orin also served as the embedded client. Clients connect to the proxy, which relays their traffic to the server, so every client–server exchange traverses the observation point. Table I summarizes the AI workloads, architectural configurations, and training datasets. Each experiment executes 100 federated learning rounds. Federated training uses a custom synchronous FedAvg implementation in PyTorch, in which the server and clients exchange length-prefixed messages over TCP; in each round, every client downloads the global model, trains locally, and uploads one update before the server aggregates and advances to the next round.

IV. Methodology

As illustrated in Fig. 1, CHAI-FP consists of passive traffic observation, multiscale feature extraction, hierarchical classification, and evaluation.

TABLE I
AI Workloads, Architectures, Variants, and Datasets

Family	Architecture	Representative Variants	Dataset
Autoencoder	Dense, Conv., VAE	Layer configurations, β-VAE, Conv. VAE	Fashion-MNIST
CNN	ResNet, MobileNet, EfficientNet	ResNet18/34/50/101, MobileNetV2/V3, EfficientNet-B0/B1/B2	Fashion-MNIST
RNN	LSTM, GRU	Single-layer, multi-layer, bidirectional variants	UCI HAR
Transformer	BERT, Distil-BERT, Tiny Transformer	BERT model sizes, lightweight configurations	AG News

A. Fingerprinting Dataset and Ground Truth

The dataset comprises about 499 million packets (670 GB, 190 hours of traffic) from 32 experiments, one per variant, each observed from three clients (Table II). Each of the 96 client traces yields one complete-trace observation and a sequence of windows; round-level quantities are available for the 90 traces whose rounds are fully reconstructed. Because larger models produce longer traces, training weights every experiment equally. Each observation is described by 68 size-normalized features covering direction and volume ratios, transmission rates, and packet-size, interarrival, burst, and idle statistics. Complete traces add nine compute-timing features derived from the round structure, four of which are cross-device ratios, giving 77 features per client and 231 in the cross-device representation (6). Labels and per-model metadata (parameter count, serialized size, and training FLOPs per sample) are computed from the model source code and are used only for evaluation.

TABLE II
Fingerprinting Dataset Composition

Family	Arch.	Var.	Traces	5 s win.	0.5 s win.
Autoencoder	3	9	27	7,943	57,786
CNN	3	9	27	35,947	298,965
RNN	2	6	18	2,578	12,036
Transformer	3	8	24	39,585	345,817
Total	11	32	96	86,053	714,604

B. Data Preprocessing

For every client connection, the proxy records the timestamp, frame length, direction, and TCP flags of each packet; capture chunks are aligned on a common timeline and mapped to client devices by address. The TCP payload is the frame length minus the header length, which is inferred per connection from pure acknowledgments: 66 bytes with the TCP timestamp option and 54 bytes without it, as on the Windows client. Transfers are recovered as maximal runs of same-direction data packets that tolerate small control packets (at most 300 B) and retransmission stalls; the dominant size class in each direction identifies the model transfers, and each upload is paired with the

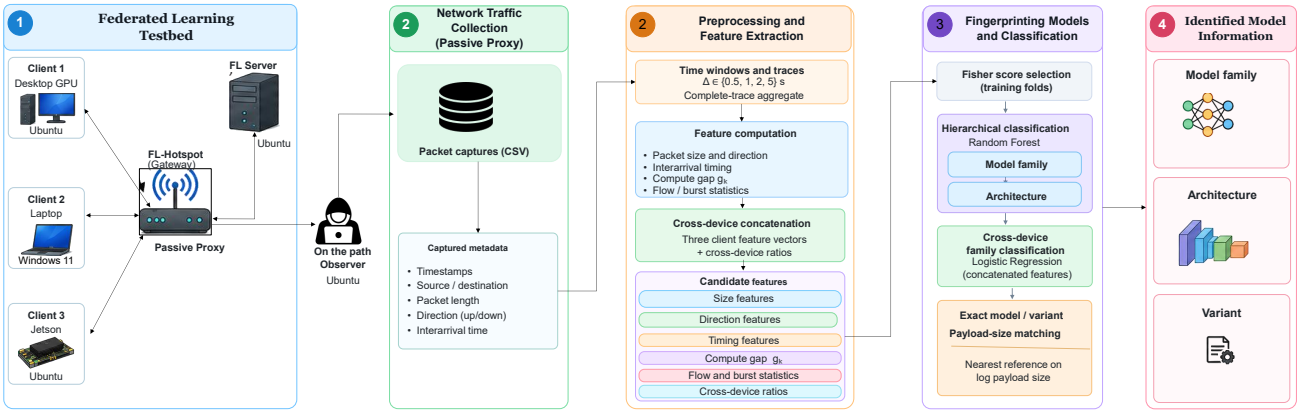


Fig. 1. Overview of the CHAI-FP hierarchical AI fingerprinting framework.

preceding download to delimit a round. Windows with fewer than 20 packets are discarded. Features that scale with model size or trace length (absolute byte and packet counts, durations, and flag counts) are excluded, missing values are set to zero for windows and to the median for traces, and features are standardized on the training folds only.

C. Traffic Observation and Feature Extraction

The proxy represents each packet by its observable metadata. Let

$$p_i = (t_i, s_i, d_i) \tag{1}$$

denote the ith observed packet, where t_i is its arrival timestamp, s_i is its size, and d_i indicates its communication direction. From the packet sequence, statistical features of sizes, interarrival times, volumes, rates, and bursts are extracted.

Beyond per-packet statistics, we exploit the round structure of federated training. Each round consists of a model download, a local-training interval with no traffic, and a model upload. From the directional packet sequence we recover, for round k, the end of the download t_k^{d} and the start of the upload t_k^{u}, and define the compute gap

$$g_k = t_k^{\mathrm{u}} - t_k^{\mathrm{d}}, \tag{2}$$

a passive proxy for local training time that excludes the inter-client synchronization wait.

D. Multiscale Traffic Representation

For an observation duration Δ, packets are grouped into windows and a feature vector is computed for each window:

$$\mathbf{x}_k^{(\Delta)} = \phi\left(\{p_i \mid k\Delta \le t_i < (k+1)\Delta\}\right), \tag{3}$$

where $\phi(\cdot)$ is the feature extraction function and $\Delta \in \{0.5, 1, 2, 5\}$ s; complete traces are also summarized as a whole.

A single short window observes only a fragment of one round. We therefore aggregate a client's window-level posteriors to a trace-level decision by averaging,

$$P(y \mid \mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} P\left(y \mid \mathbf{x}_k^{(\Delta)}\right), \tag{4}$$

where $\mathcal{T}$ indexes the windows of a trace.

For cross-device window classification, the three clients' windows covering the same global interval are concatenated, analogously to (6). Near-idle windows, whose transferred bytes fall below 5% of the trace's busy-window volume or whose retransmission share exceeds 5%, are removed before classification; this filter is fixed in advance and uses no labels. For short windows, variant identity is assessed by nearest-reference matching on the size of a model download that completes within the window. To study observation times beyond the window length, the posteriors of consecutive 5 s windows are averaged with (4) over non-overlapping spans of 10 to 30 s.

E. Hierarchical AI Model Fingerprinting

Model identity is represented by a three-level hierarchy $\mathcal{H} = \mathcal{F} \to \mathcal{A} \to \mathcal{V}$ of families, architectures, and variants. Given a fingerprint $\mathbf{x}$, each level is conditioned on the one above:

$$\begin{gathered}\hat{f} = \arg\max_{f \in \mathcal{F}} P(f \mid \mathbf{x}), \quad \hat{a} = \arg\max_{a \in \mathcal{A}(\hat{f})} P(a \mid \mathbf{x}, \hat{f}), \\ \hat{v} = \arg\max_{v \in \mathcal{V}(\hat{a})} P(v \mid \mathbf{x}, \hat{a}),\end{gathered} \tag{5}$$

where $\mathcal{A}(\hat{f})$ and $\mathcal{V}(\hat{a})$ are the architectures of the selected family and the variants of the selected architecture. To isolate per-level discriminability, each level is also evaluated with the ground-truth parent.

F. Cross-Device Fingerprinting

In federated learning, the participating clients train the same model on heterogeneous hardware and operating systems. Classifying each client in isolation discards the most

discriminative cue available to a passive observer: how the timing and transfer shape of the identical model differ across devices, which is governed by the architecture's per-layer compute profile. We therefore represent each experiment e by a single feature vector that concatenates the per-device summaries,

$$\mathbf{x}_e = \left[\, \mathbf{f}_e^{(1)} \,\|\, \mathbf{f}_e^{(2)} \,\|\, \mathbf{f}_e^{(3)} \,\right], \tag{6}$$

where $\mathbf{f}_e^{(c)} \in \mathbb{R}^{77}$ represents client c in experiment e. Each vector contains 68 traffic features and nine compute-timing features, including four shared cross-device ratios. Consequently, $\mathbf{x}_e \in \mathbb{R}^{231}$ contains 223 nonredundant dimensions. Because (6) yields few high-dimensional examples, family identification (5) is performed with an ℓ_2-regularized multinomial logistic regression whose regularization strength is chosen by nested cross-validation (Sec. IV-G); tree ensembles overfit this regime.

G. Feature Selection, Classification, and Validation

Features are ranked by Fisher score, the between-class separation relative to within-class variability, independently within each training fold. For the per-client representations we use Fisher-selected random forests; for the cross-device representation (6) we use regularized logistic regression. Exact-model identification is performed by nearest-reference matching on the logarithm of the uploaded payload size.

Generalization is assessed under experiment-disjoint, leave-one-model-out validation, in which every held-out model is absent from training, so performance measures recognition of unseen models rather than memorization. All hyperparameter and feature-selection choices are made within an inner loop over the training models, so held-out executions never influence model selection. Performance is reported as balanced accuracy, whose chance level is $1/|\mathcal{C}|$ for a $|\mathcal{C}|$-class task, independent of class sizes; reliability is quantified by a group-bootstrap 95% confidence interval over models and a label-permutation test against chance.

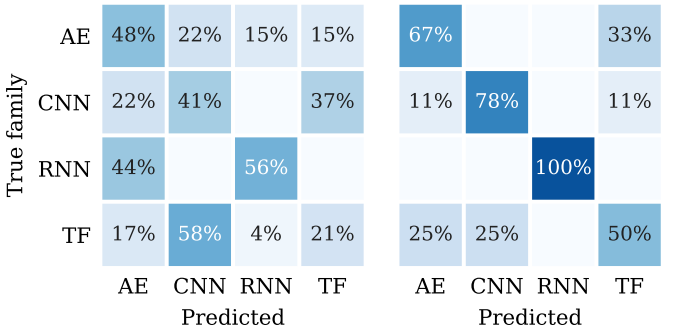
Robustness to size concealment is assessed by simulating a padding defense in which every transfer is padded up to a size bucket, and by repeating family identification using only features independent of transfer size: the compute gap, its dispersion, and cross-device gap ratios.

V. Results

We report fingerprinting performance at the family, architecture, and variant levels, then for short observation windows and under a size-padding defense. Scores are balanced accuracy, for which chance is $1/|\mathcal{C}|$.

A. Family Identification

Table III compares family identification methods. Under the repeated protocol, the cross-device representation achieves 0.68 ± 0.04, the highest among all methods (fixed late fusion: 0.61) and 0.20 above a per-client random forest (0.48 ± 0.04). Under leave-one-model-out validation, which trains on all remaining models, it reaches 0.74 (95% CI 0.58–0.88, macro F1 0.74), significantly above chance (permutation test, $p = 0.003$), compared with 0.41 (95% CI 0.29–0.56) for a single-client random forest that observes each client in isolation. Per-client classification that also receives the four shared cross-device ratios reaches 0.58, and counting each ratio once in the cross-device vector gives 0.76. Recurrent networks are identified without error (6/6), convolutional networks in 7 of 9 cases, autoencoders in 6 of 9, and transformers in 4 of 8 (Fig. 2). The remaining errors involve architectures whose compute per parameter overlaps.



(a) Single-client RF (0.41) (b) Cross-device LR (0.74)

Fig. 2. Family confusion matrices for (a) the single-client baseline and (b) the cross-device representation (leave-one-model-out). Cells give the percentage of each true family assigned to each predicted family; empty cells are zero. Balanced accuracy in parentheses.

TABLE III
Family Identification (4 classes, chance 0.25)

	Balanced accuracy	
Method	Repeated CV	Leave-one-model-out
Fixed late fusion	0.61	–
Equal-probability hybrid fusion	0.68*	–
Per-client RF (shared ratios)	0.48	0.58
Single-client RF	–	0.41
Cross-device LR (proposed)	0.68	0.74

Repeated CV: mean over 20 seeds × 5-fold model-disjoint cross-validation. 95% CI 0.58–0.88 for the proposed method. *Single 5-fold run.

B. Architecture Identification

Using packet-morphology features, the architecture is identified within each family well above chance (Table IV). Recurrent architectures are separated perfectly (1.00), autoencoder architectures reach 0.89, and convolutional architectures 0.78. Transformer architectures are the most difficult (0.47), partly because DistilBERT is represented by a single model. The mean across families is 0.79 (macro F1 0.78), and the cross-device representation raises convolutional architectures further to 0.81.

C. Variant Identification

At the finest level of the hierarchy, the uploaded payload size provides a baseline leak. It is recovered from traffic

TABLE IV
Architecture Identification Within Each Family (Balanced Accuracy)

Method	AE	CNN	RNN	Transf.	Mean
Per-client RF (packet)	0.89	0.78	1.00	0.47	0.79
Cross-device LR	0.67	0.81	1.00	0.25	0.68
Chance	0.33	0.33	0.50	0.33	–

within a median relative error of 0.025% of the true serialized size. Because every variant has a distinct size, nearest-reference matching identifies the exact variant in all 90 traces (chance 3.4%). Table V summarizes the result per family. Accounting for the client-specific header length is essential: assuming a single header length under-counts the Windows client's payload by 0.8% and causes three of the 90 identifications to fail. Within a 5 s window, every variant with a short download is identified, including all recurrent networks, autoencoders, MobileNets, and Tiny Transformers, whereas EfficientNet, ResNet, and larger BERT downloads exceed the window.

TABLE V
Variant Identification by Payload Size

Family	Traces	Payload (MB)	Trace	5 s
Autoencoder	27	0.01–10.1	1.00	0.96
CNN	24	6.2–170.7	1.00	0.25
RNN	18	0.22–2.2	1.00	1.00
Transformer	21	6.8–373.8	1.00	0.57
Total	90		1.00	0.69

Nearest-reference matching on log payload size; 5 s: one complete download within a 5 s window. ResNet-34 and DistilBERT are excluded (incomplete capture).

D. Discriminative Features

Within training folds, transmission rates rank highest for per-client family identification, burst and packet-size dispersion for the cross-device representation, and compute-timing features for architecture identification in three of the four families.

E. Short Observation Windows

Table VI reports all three levels for single windows of 0.5 to 5 s and for longer observation times of 10 to 30 s. Single-client windows reach only 0.26–0.31 for family, whereas the cross-device windows reach 0.41 at 0.5 s and 0.51 at 5 s, and 0.51–0.65 after averaging posteriors per experiment. Convolutional architectures are identified with 0.65–0.71 per window, whereas recurrent and transformer architectures, each represented by few models, are not. A window that begins at a download identifies the variant whenever the download completes within it, which holds for 23% of traces at 0.5 s and 69% at 5 s.

Accumulating evidence over longer observation helps where the per-window signal is consistent (Table VI). Averaging the posteriors of consecutive 5 s windows raises cross-device family accuracy from 0.51 at 5 s to 0.56 at 30 s

TABLE VI
Accuracy Versus Observation Time

	Single window				Accumulated			
Task	0.5 s	1 s	2 s	5 s	10 s	15 s	20 s	30 s
Family, single client	0.26	0.29	0.30	0.31	0.31	0.31	0.30	0.30
Family, cross-device	0.41	0.43	0.46	0.51	0.53	0.54	0.55	0.56
Arch., autoencoder	0.50	0.52	0.45	0.47	0.48	0.48	0.48	0.47
Arch., CNN	0.65	0.67	0.69	0.71	0.76	0.78	0.80	0.81
Arch., RNN	0.54	0.51	0.39	0.35	0.29	0.29	0.27	0.28
Arch., transformer	0.21	0.22	0.20	0.19	0.20	0.19	0.19	0.19
Variant†	0.23	0.32	0.39	0.69	0.80	0.82	0.86	0.90

Balanced accuracy per decision. Family and architecture: cross-device LR with near-idle windows removed, experiment-disjoint 5-fold CV grouped by model; 10–30 s average the posteriors of consecutive 5 s windows over non-overlapping spans. Chance 0.25 (family), 0.33 (autoencoder, CNN, transformer), 0.50 (RNN). †Share of traces whose variant is identified from one complete download within the observation time.

and convolutional architecture accuracy from 0.71 to 0.81, whereas the single-client baseline remains at 0.30–0.31 and autoencoder architecture does not improve. Variant identification, which depends on capturing a complete download, rises from 69% of traces at 5 s to 90% at 30 s.

F. Ablation

Table VII isolates the components of the proposed model under the same leave-one-model-out protocol with nested selection. Neither component helps in isolation: the cross-device representation with a random forest reaches 0.45, and a per-client logistic regression also reaches 0.45, both below the per-client random forest with shared ratios (0.58), and a strictly single-client random forest reaches 0.41. Combined, they reach 0.74. Concatenating the three devices yields few high-dimensional examples, which tree ensembles overfit but a regularized linear model exploits. The compute-timing features contribute a further 0.15 (0.59 without them). For short windows, the cross-device representation improves both classifiers (5 s per-window accuracy from 0.31 to 0.44 with a random forest and from 0.35 to 0.49 with logistic regression), and removing near-idle windows adds 0.02 per window. The cross-device gain does not depend on a single device: with the laptop removed, combining the remaining two devices still reaches 0.71 (95% CI 0.57–0.85). Excluding the ten variants whose compute profiles overlap another family raises accuracy to 0.85, an upper bound rather than an estimate of generalization.

G. Robustness to Size Padding

Padding every transfer to a size bucket defeats variant identification: accuracy falls from 1.00 to 0.60 with 1 MB buckets (1.8% byte overhead), to 0.45 with power-of-two buckets (47%), and to chance (0.03) when all transfers are padded to the largest model, at 1,086% overhead. Family identification is unaffected, because the compute gap and cross-device timing ratios do not depend on transfer size: using these timing features alone, cross-device family identification reaches 0.62 (95% CI 0.45–0.78, $p = 0.005$), matching the same model with size fea-

TABLE VII
Ablation of the Family Model (Balanced Accuracy)

Representation	Classifier	Comp.	Bal. acc.	95% CI
Complete trace (leave-one-model-out)				
Per-client	RF	✓	0.58	0.42–0.74
Per-client	LR	✓	0.45	0.34–0.56
Single-client	RF	✓	0.41	0.29–0.56
Cross-device	RF	✓	0.45	0.27–0.64
Cross-device	LR	–	0.59	0.45–0.75
Cross-device	LR	✓	0.74	0.58–0.88
5 s windows, per window / pooled				
Single-client	RF	–	0.31 / 0.31	
Single-client	LR	–	0.35 / 0.40	
Cross-device	RF	–	0.44 / 0.45	
Cross-device	LR	–	0.49 / 0.62	
Cross-device†	LR	–	0.51 / 0.62	

Comp.: compute-timing features. Per-client rows include the shared cross-device ratios. †Near-idle windows removed. Windows: experiment-disjoint 5-fold CV grouped by model.

tures (0.60). Concealing model identity therefore requires timing defenses in addition to size padding.

VI. Countermeasures

Size padding removes the variant signal (Sec. V-G) but at a high cost: padding to the largest model adds 1,086% traffic, whereas inexpensive 1 MB buckets still allow 60% identification. Because family leakage stems from timing, defenses must decouple traffic timing from local computation, for example through deadline-based rounds with fixed upload times, randomized upload delays, cover traffic during local training, or relays that batch uploads from several clients. Encryption and secure aggregation protect update contents but leave transfer timing unchanged.

VII. Conclusion

This paper presented CHAI-FP, a hierarchical framework that characterizes how much AI model identity leaks from the network traffic metadata of federated learning. A passive observer that sees only packet timing, size, and direction can reconstruct the FL round structure, including the compute gap between model download and upload, and can combine the traffic of clients that train the same model on heterogeneous hardware and operating systems. On a physical testbed with Ubuntu and Windows clients, this cross-device fingerprinting identifies the family of unseen models with a balanced accuracy of 0.74 from complete traces, compared with 0.41 for single-client classification, and 0.51 from a single 5 s window, rising to 0.56 over 30 s of observation. Within a family, the architecture is identified with a mean balanced accuracy of 0.79, and the uploaded payload size reveals the exact variant in every trace and in 69% of traces from one 5 s window. Size padding removes the variant signal but not the timing signal that reveals the family, indicating that protecting model identity in FL requires timing defenses in addition to size concealment. Future work will extend the evaluation to additional architectures and repeated executions, to encrypted transports and frameworks such as Flower over gRPC, and to varying training configurations, and will quantify the privacy and latency trade-offs of timing defenses.

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," Proceedings of the IEEE, vol. 107, no. 8, pp. 1738–1762, 2019.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. of the 20th International Conference on Artificial Intelligence and Statistics, ser. Proc. of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[3] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in Proc. of the ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 1928–1943.

[4] M. N. H. Shuvo and M. Hossain, "Fingerprinting deep learning models via network traffic patterns in federated learning," in Proc. of the ACM Workshop on Wireless Security and Machine Learning, 2025, pp. 32–37.

[5] W. Hua, Z. Zhang, and G. E. Suh, "Reverse engineering convolutional neural networks through side-channel information leaks," in Proc. of the 55th ACM/ESDA/IEEE Design Automation Conference (DAC), 2018, pp. 1–6.

[6] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache telepathy: Leveraging shared resource attacks to learn DNN architectures," in Proc. of the 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, 2020, pp. 2003–2020.

[7] X. Hu, L. Liang, S. Li, L. Deng, P. Zuo, Y. Ji, X. Xie, Y. Ding, C. Liu, T. Sherwood, and Y. Xie, "DeepSniffer: A DNN model extraction framework based on learning architectural hints," in Proc. of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, 2020, pp. 385–399.

[8] K. Patwari, S. M. Hafiz, H. Wang, H. Homayoun, Z. Shafiq, and C. N. Chuah, "DNN model architecture fingerprinting attack on CPU GPU edge devices," in Proc. of the IEEE European Symposium on Security and Privacy, 2022, pp. 337–355.

[9] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in Proc. of the 10th ACM Workshop on Privacy in the Electronic Society, 2011, pp. 103–114.

[10] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in Proc. of the 23rd USENIX Security Symposium, 2014, pp. 143–157.

[11] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in Proc. of the 25th USENIX Security Symposium, 2016, pp. 1187–1203.

[12] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in Proc. of the Network and Distributed System Security Symposium, 2018.

[13] J. Li, S. Wu, H. Zhou, X. Luo, T. Wang, Y. Liu, and X. Ma, "Packetlevel open-world app fingerprinting on wireless traffic," in Proc. of the Network and Distributed System Security Symposium (NDSS'22), 2022.

[14] M. N. H. Shuvo, M. Hossain, A. Mallik, J. N. Twigg, and F. T. Dagefu, "FLARE: A wireless side channel fingerprinting attack on federated learning," in Proc. of the IEEE Conference on Computer Communications, 2026, pp. 1–10.

[15] L. Oyaniyi, A. Mallik, M. Hossain, and W. Yu, "Fingerprinting AI applications and phases in edge assisted distributed learning and inference using side channel data," in Proc. of the IEEE International Conference on Communications, 2026, pp. 1–6.

[16] M. N. H. Shuvo, M. H. Ashik, and M. Hossain, "FLINT: Fingerprinting federated learning architectures from 5G PHY-layer side channels," arXiv preprint arXiv:2607.15469, 2026.